Université Ibn Zohr
Ecole Supérieure de Technologie – Guelmim
Département d'Informatique

A.U. 2025-2026
DUT ID/S3

TD : Initiation à la complexité algorithmique en temps

- **Exercice 1**

```python
def get_first_element(arr):
    return arr[0]
```

Question :

1. Quelle est la complexité temporelle ?
2. Pourquoi ne dépend-elle pas de la taille de la liste ?

- **Exercice 2**

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Question :

1. Pourquoi la recherche devient deux fois plus rapide à chaque itération ?
2. Quelle est la complexité temporelle ?

- **Exercice 3**

```python
def find_max(arr):
    max_val = arr[0]
    for v in arr:
        if v > max_val: max_val = v
    return max_val
```

Question :

1. Combien de fois la boucle s'exécute-t-elle pour une liste de taille n ?
2. Quelle est la complexité ?

- **Exercice 4**

```python
def quick_sort(arr):
    if len(arr) <= 1: return arr
    pivot = arr[len(arr)//2]
    left  = [x for x in arr if x < pivot]
    mid   = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + mid + quick_sort(right)
```

Question :

1. Pourquoi cet algorithme est plus efficace que le tri à bulles ?
2. Quelle est sa complexité moyenne et pourquoi ?

- **Exercice 5**

```
def bubble_sort(arr):
    for i in range(len(arr)):
        for j in range(len(arr)-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

Question :

1. Pourquoi y a-t-il deux boucles imbriquées ?
2. Quelle est la complexité en fonction de n ?

- **Exercice 6**

```
def fibonacci(n):
    if n <= 1: return n
    return fibonacci(n-1) + fibonacci(n-2)
```

Question :

1. Combien d'appels récursifs pour n = 5 ?
2. Pourquoi le temps explose très vite ?

- **Exercice 7**

```
import itertools
def generate_permutations(arr):
    return list(itertools.permutations(arr))
```

Question :

1. Combien de permutations pour une liste de 4 éléments ?
2. Quelle est la complexité ?

Travail demandé

1. Pour chaque algorithme, exécutez-le avec différentes tailles de listes (n = 10, 20)
2. Notez le **temps d'exécution mesuré** avec time.time()
3. Comparez le résultat avec la **complexité théorique**
4. Rédigez une **conclusion** sur comment la taille n influence la performance.